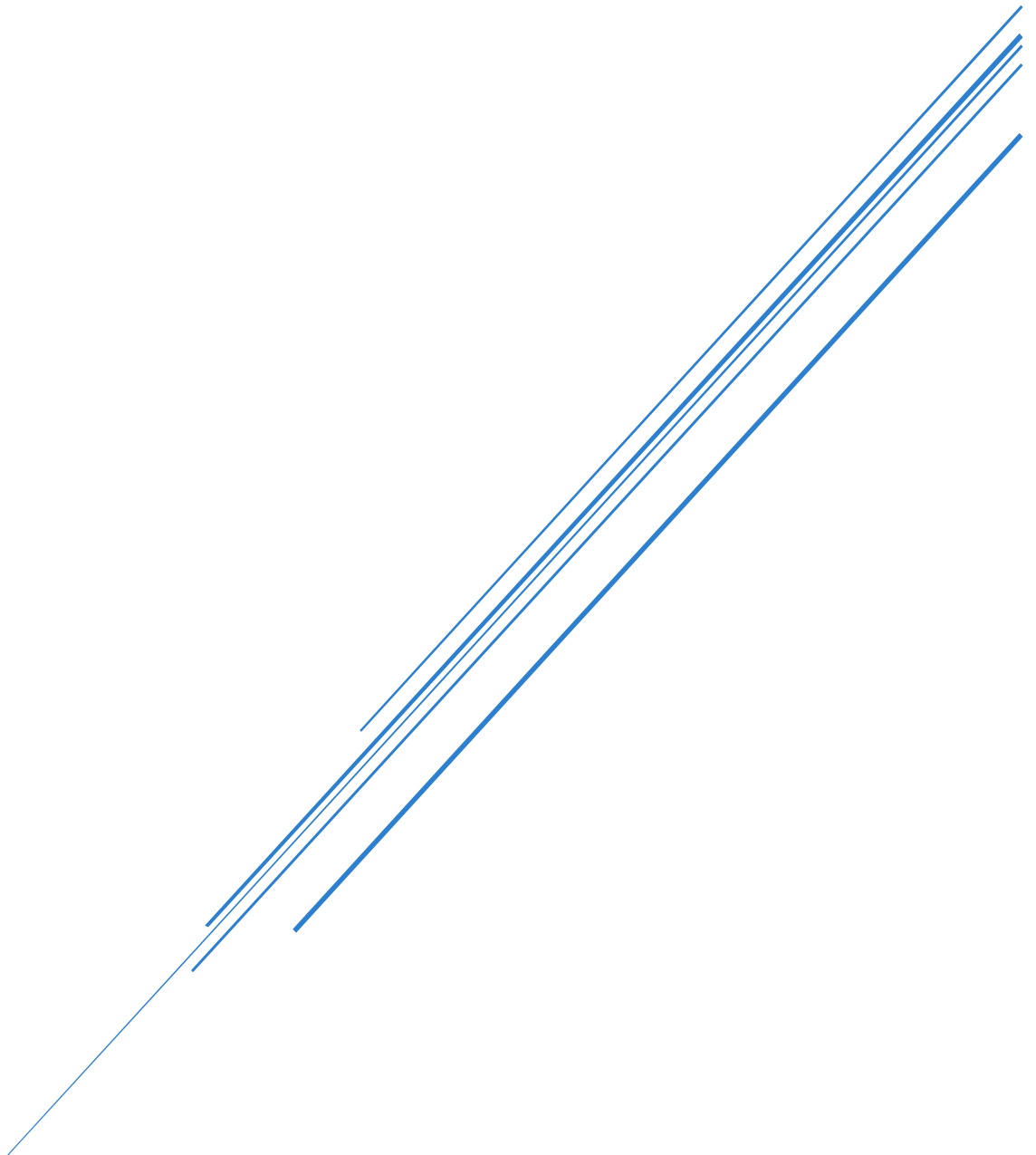


SQL JOINS EXERCISE

NAME: BOTSHELO MOILWENYANE

STUDENT NUMBER: 2805001389

COURSE: DATA ANALYTICS WITH SQL



This exercise was completed using the Databricks Spark SQL environment. Tables were created and populated within Databricks, and all queries were written and executed using Spark SQL to retrieve and validate the data. The platform provided an efficient environment for performing the SQL joins exercise.

Below is the code that I used to create tables on Databricks.

```
-Creating table users

Create Table users (
  user_id INTEGER,
  user_name STRING,
  country STRING
);

INSERT INTO users (
  user_id,
  user_name,
  country
)
VALUES
(1, 'Nomvula', 'Johannesburg'),
(2, 'David', 'Cape town'),
(3, 'Anele', 'Durban'),
(4, 'Kabelo', 'Pretoria'),
(5, 'Lerato', 'Port Elizabeth');

--Creating table plans
Create table plans (
  plan_id BIGINT,
  plan_name STRING,
  monthly_price BIGINT
);

INSERT INTO plans (
  plan_id,
  plan_name,
  monthly_price
)
VALUES
(10, 'Basic', 99),
(11, 'Standard', 129),
(12, 'Premium', 199),
(13, 'Family', 249),
(14, 'Mobile', 59);
```

```
--Creating table subscriptions
```

```
Create table subscriptions (  
    subscription_id INTEGER,  
    user_id INTEGER,  
    plan_id INTEGER,  
    start_date DATE  
);
```

```
INSERT INTO subscriptions (  
    subscription_id,  
    user_id,  
    plan_id,  
    start_date  
)
```

```
VALUES
```

```
(501, 1, 10, '2026-01-15'),  
(502, 2, 11, '2026-02-01'),  
(503, 1, 12, '2026-03-10'),  
(504, 6, 11, '2026-03-20'),  
(505, 3, 13, '2026-04-05');
```

```
--Creating table shows
```

```
Create table shows (  
    show_id INTEGER,  
    show_title STRING,  
    genre STRING  
);
```

```
INSERT INTO shows(  
    show_id,  
    show_title,  
    genre  
)
```

```
VALUES
```

```
(701, 'Comedy Hour', 'Comedy'),  
(702, 'Crime Time', 'Drama'),  
(703, 'Tech Tales', 'Documentary'),  
(704, 'Cooking Lab', 'Lifestyle'),  
(706, 'Wild Earth', 'Documentary');
```

```
--Create table viewing_sessions
```

```

Create table viewing_sessions (
    session_id INTEGER,
    user_id INTEGER,
    show_id INTEGER,
    watch_minutes INTEGER
);

INSERT INTO viewing_sessions(
    session_id,
    user_id,
    show_id,
    watch_minutes
)
VALUES
(901, 1, 701, 45),
(902, 2, 703, 30),
(903, 1, 702, 60),
(904, 7, 701, 20),
(905, 3, 705, 90);

```

Below is the code that I used for the SQL JOINS questions.

```

--Q1: Show every user who has a subscription. Match users to subscriptions.

SELECT A.user_id,
       A.user_name,
       B.subscription_id,
       B.start_date
FROM users As A
INNER JOIN subscriptions As B
ON A.user_id = B.user_id;

--Q2: Show every subscription with its matching plan name name and monthly price

SELECT A.subscription_id,
       A.user_id,
       B.plan_name,
       B.monthly_price
FROM subscriptions As A
INNER JOIN plans AS B
ON A.plan_id = B.plan_id;

```

--Q3 Show every viewing session that has a matching show. Include the show title and genre

```
SELECT A.session_id,  
       B.show_title,  
       B.genre,  
       A.watch_minutes  
FROM viewing_sessions As A  
INNER JOIN shows As B  
ON A.show_id = B.show_id;
```

--Q4 Show every viewing session with the user who watched it. Show sessions with a matching user

```
SELECT A.user_name,  
       A.country,  
       B.session_id,  
       B.show_id,  
       B.watch_minutes  
FROM users As A  
INNER JOIN viewing_sessions As B  
ON A.user_id = B.user_id;
```

--Q5 Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan.

```
SELECT A.user_name,  
       A.country,  
       C.plan_name,  
       C.monthly_price,  
       B.start_date  
FROM users As A  
INNER JOIN subscriptions As B  
ON A.user_id = B.user_id  
INNER JOIN plans As C  
ON B.plan_id = C.plan_id;
```

--Q6 Show every user and any subscriptions they have. Users without subscriptions must still appear.

```
SELECT A.user_id,  
       A.user_name,  
       B.subscription_id,  
       B.start_date  
FROM users As A  
LEFT JOIN subscriptions As B
```

```
ON A.user_id = B.user_id;
```

--Q7 Show every plan and the subscriptions on it. Plans with no subscribers must still appear.

```
SELECT A.plan_id,  
       A.plan_name,  
       B.subscription_id,  
       B.user_id  
FROM plans As A  
LEFT JOIN subscriptions As B  
ON A.plan_id = B.plan_id;
```

--Q8 Show every show and any viewing sessions on it. Shows that were never watched must still appear.

```
SELECT A.show_id,  
       A.show_title,  
       B.session_id,  
       B.watch_minutes  
FROM shows As A  
LEFT JOIN viewing_sessions As B  
ON A.show_id = B.show_id;
```

--Q9 Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear(with NULL user details)

```
SELECT A.session_id,  
       A.show_id,  
       A.watch_minutes,  
       B.user_id,  
       B.user_name  
FROM viewing_sessions As A  
LEFT JOIN users As B  
ON A.user_id = B.user_id;
```

--Q10 Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear.

```
SELECT A.user_name,  
       A.country,  
       C.plan_name,  
       C.monthly_price  
FROM users As A  
LEFT JOIN subscriptions As B  
ON A.user_id = B.user_id
```

```
LEFT JOIN plans As C
ON B.plan_id = C.plan_id;
```

--Q11 Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist.

```
SELECT A.user_id,
       A.user_name,
       B.subscription_id,
       B.start_date
FROM users As A
FULL OUTER JOIN subscriptions As B
ON A.user_id = B.user_id;
```

--Q12 Show every plan and every subscription, including plans without subscribers AND any subscription referencing plan that does not exist.

```
SELECT A.plan_id,
       A.plan_name,
       B.subscription_id,
       B.user_id
FROM plans As A
FULL OUTER JOIN subscriptions As B
ON A.plan_id = B.plan_id;
```

--Q13 Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist.

```
SELECT A.show_id,
       A.show_title,
       B.session_id,
       B.watch_minutes
FROM shows As A
FULL OUTER JOIN viewing_sessions As B
ON A.show_id = B.show_id;
```

--Q14 Show every user and every viewing session, including users with no sessions AND session referencing users who do not exist.

```
SELECT A.user_id,
       A.user_name,
       B.session_id,
       B.show_id,
       B.watch_minutes
FROM users As A
FULL OUTER JOIN viewing_sessions As B
```

```
ON A.user_id = B.user_id;
```

--Q15 Show every user, every subscription, and every plan in one query-using FULL OUTER JOIN throughout.

```
SELECT A.user_id,  
       A.user_name,  
       B.subscription_id,  
       B.plan_id,  
       C.plan_name  
FROM users As A  
FULL OUTER JOIN subscriptions As B  
ON A.user_id = B.user_id  
FULL OUTER JOIN plans As C  
ON B.plan_id = C.plan_id;
```